

GUÍA DEFINITIVA DE ESTUDIO: TEORÍA DE PROGRAMACIÓN I (C++) - UTN

1. Estructura Básica de un Programa y Librerías

- **#include <iostream>**: Librería estándar que permite la entrada y salida de datos a través de la consola.
- **#include <string>**: Librería obligatoria para poder declarar y manipular variables de texto complejas.
- **using namespace std;**: Directiva que indica al compilador que use el espacio de nombres estándar. Evita escribir el prefijo `std::` antes de comandos como `cout`, `cin` o `string`.
- **int main()**: Función principal y punto de entrada obligatorio de cualquier programa en C++. Aquí inicia y termina la ejecución.
- **return 0;**: Sentencia al final de `main` que retorna el valor 0 al sistema operativo, indicando que el programa finalizó con éxito y sin errores.

2. Variables, Constantes y Gestión de Memoria

- **Variable**: Espacio reservado en la memoria RAM cuyo valor puede cambiar a lo largo de la ejecución del programa.
- **Basura en memoria**: Una variable creada sin un valor inicial (Ej: `int x;`) contiene datos viejos e impredecibles que quedaron en esa dirección de memoria. Puede causar errores graves si se usa sin inicializar.
- **Constante**: Espacio de memoria de solo lectura cuyo valor se define al inicio y no puede ser alterado jamás.
- **Sintaxis de constantes**: Se define anteponiendo la palabra clave `const` (Ej: `const int MAX = 10;`). Por buena práctica, sus nombres se escriben en mayúsculas.
- **Tipos de datos fundamentales**:
 - `int`: Almacena números enteros (positivos y negativos) sin decimales (ocupa generalmente 4 bytes).
 - `float`: Almacena números con punto decimal de precisión simple (ocupa 4 bytes).
 - `double`: Almacena números con punto decimal de precisión doble (ocupa 8 bytes).
 - `char`: Almacena un único carácter o letra entre comillas simples (Ej: `'A'`).
 - `bool`: Almacena valores lógicos de verdad: `true` (representado internamente como 1) o `false` (representado como 0).

3. Entrada, Salida y Control de Consola

- **cout:** Objeto de flujo de salida usado para mostrar datos en la pantalla mediante el operador de inserción `<<` (Ej: `cout << "Hola";`).
- **cin:** Objeto de flujo de entrada usado para capturar datos desde el teclado mediante el operador de extracción `>>` (Ej: `cin >> variable;`).
- **endl:** Manipulador que inserta un salto de línea en la consola y limpia el búfer de salida.
- **Lectura de strings con `cin >>`:** Lee texto del teclado pero se detiene al encontrar el primer espacio en blanco, tabulación o salto de línea. No sirve para frases completas.
- **getline(cin, variable):** Función ideal para leer una línea completa de texto incluyendo todos sus espacios en blanco.
- **Pausa estándar (`cin.get()`):** Detiene el programa hasta que el usuario presiona Enter. Si antes se usó `cin >>`, se debe escribir `cin.ignore();` justo antes para borrar el Enter que quedó atrapado en la memoria intermedia (búfer).
- **Pausa no estándar (`system("pause")`):** Comando exclusivo de Windows. No es portable y los profesores de la UTN la consideran una mala práctica de programación.

4. Estructuras Condicionales y Operadores

- **Asignación (`=`):** El signo `=` copia el valor que está del lado derecho y lo guarda dentro de la variable de la izquierda.
- **Comparación (`==`):** El doble signo `==` verifica si dos valores son iguales. Confundirlo con el `=` en un condicional es un error típico de examen que cambia el valor de la variable en lugar de compararla.
- **Operadores relacionales:** Permiten comparar valores matemáticos (`==`, `!=`, `<`, `>`, `<=`, `>=`).
- **Estructura if / else:** Bifurcación condicional. El bloque `else` es opcional y se ejecuta de forma obligatoria únicamente si la condición del `if` resulta ser falsa (`false`).
- **Llaves obligatorias `{}`:** Las llaves se exigen si el bloque del `if` o del `else` debe ejecutar más de una línea de código. Si hay una sola línea, las llaves son opcionales.

5. Operadores Lógicos y Evaluación en Cortocircuito

- **&& (Operador Y / AND):** Devuelve verdadero solo si **todas** las condiciones evaluadas son verdaderas al mismo tiempo.
- **|| (Operador O / OR):** Devuelve verdadero si **al menos una** de las condiciones evaluadas es verdadera.

- **! (Operador NO / NOT):** Operador unario que invierte el valor de verdad lógico (Ej: `!true` se transforma en `false`).
- **Evaluación en cortocircuito:** C++ optimiza el tiempo de ejecución evaluando de izquierda a derecha y deteniéndose cuando el resultado final ya es seguro:
 - En un `&&`, si la primera condición da falso, C++ **no evalúa** la segunda porque ya sabe que todo el conjunto será falso.
 - En un `||`, si la primera condición da verdadero, C++ **no evalúa** la segunda porque ya sabe que todo el conjunto será verdadero.

6. Estructuras de Repetición (Bucles)

- **while:** Bucle que evalúa la condición **antes** de entrar al bloque. Si la condición es falsa desde el principio, el bloque de código se ejecuta cero (0) veces.
- **do-while:** Bucle que ejecuta el bloque de código **primero** y evalúa la condición al final. Garantiza que el código se ejecute al menos una (1) vez. **¡Atención!** Lleva punto y coma al final de la condición: `while(condición);`. `while (cont > 0) {}`
- **for:** Bucle ideal para usar cuando conoces de antemano la cantidad exacta de repeticiones a realizar. Estructura: `for(inicialización; condición; incremento)`.
- **Alcance de la variable (Scope):** Si declaras la variable de control dentro del `for` (Ej: `for(int i = 0; ...)`), esa variable `i` solo existe dentro del bucle. Intentar usarla afuera provocará un error de compilación.
- **Contador:** Variable que incrementa o decrementa en un valor constante fijo (Ej: `c++` o `c = c + 1`).
- **Acumulador:** Variable que suma valores variables para obtener un total acumulado (Ej: `suma = suma + sueldo`).
- **Asignación compuesta:** Operadores abreviados como `+=`, `-=`, `*=`, `/=`. Escribir `acumulador += sueldo;` es exactamente lo mismo que poner `acumulador = acumulador + sueldo;`.

7. Arreglos (Arrays / Vectores Unidimensionales)

- **Definición técnica:** Colección de datos estática, homogénea y contigua en memoria. Es *estática* porque su tamaño se define al compilar y no cambia; *homogénea* porque todos los elementos son estrictamente del mismo tipo de dato; y *contigua* porque se reservan casilleros de memoria uno al lado del otro.
- **Declaración vacía:** `tipo nombre[TAMANO];` (Ej: `int numeros[5];`). Reserva el espacio, pero todos sus casilleros contienen "basura".

- **Declaración con inicialización:** `tipo nombre[TAMANO] = {valores};` (Ej: `int notas[3] = {8, 9, 4};`).
- **Inicialización completa en cero:** `tipo nombre[TAMANO] = {0};` pone absolutamente todas las posiciones del array en cero de forma automática.
- **Tamaño implícito:** `int valores[] = {1, 2, 3};` Si dejas los corchetes vacíos pero pasas los datos, C++ calcula automáticamente que el tamaño del vector es 3.
- **Restricción de tamaño (Regla de oro):** No puedes usar una variable común para definir el tamaño de un array estándar (Ej: `int n = 5; int vec[n];` **está mal**). El tamaño debe ser un número entero literal o una constante definida previamente (`const int TAM = 5; int vec[TAM];`).
- **Índice:** Posición numérica de un elemento dentro del vector. Arranca siempre de forma obligatoria en la posición `0`.
- **Última posición válida:** El último elemento se ubica siempre en el índice `tamaño - 1` (Ej: si el array es de tamaño 5, las posiciones válidas van del `0` al `4`).
- **Desborde de Array (Buffer Overflow):** Intentar acceder o escribir en un índice inexistente (Ej: usar `numeros[5]` en un array de tamaño 5) rompe el programa o invade memoria de otras variables, ya que C++ no controla los límites en tiempo de ejecución.
- **Recorrido estándar con `for`:** Para recorrer un array completo, el bucle inicia en `0` y su condición debe ser estrictamente menor que el tamaño:

cpp

```
const int TAM = 5;
int miArray[TAM] = {10, 20, 30, 40, 50};
for(int i = 0; i < TAM; i++) {
    cout << miArray[i] << " "; // Imprime las posiciones 0, 1, 2, 3 y 4
}
```

- **Fórmula automática de tamaño:** Para averiguar cuántos elementos tiene un array se utiliza:

cpp

```
int tamaño = sizeof(numeros) / sizeof(numeros[0]);
```

- `sizeof(numeros)` devuelve el tamaño total de todo el vector medido en bytes.
- `sizeof(numeros[0])` devuelve el tamaño en bytes de un único elemento (el primero), según su tipo de dato.